

UNIT-5 | Lecture 4

Join Strategies

Coverage: Hour 4

Learning Objectives

Understand common join algorithms.

Know when different join strategies are useful.

Nested-Loop Join

For each tuple/page of one relation, scan the other relation to find matching tuples.

Simple nested-loop: compares tuples directly.

Block nested-loop: processes blocks to reduce repeated I/O.

Indexed Nested-Loop Join

Uses an index on the inner relation's join attribute.

For each outer tuple, the index can quickly locate matching inner tuples.

Useful when the inner relation has an appropriate index and the outer input is reasonably small/selective.

Sort-Merge Join

Sort both relations on the join attribute, then scan the sorted relations to find matches.

Useful when inputs are already sorted or sorting cost is acceptable, and it can work well for large relations.

Hash Join

Build a hash table on one relation using the join attribute, then probe it with tuples from the other relation.

Often efficient for equality joins when sufficient memory is available.

Comparison

Strategy	Main Idea
Nested-loop	Compare each outer tuple/block with inner input
Indexed nested-loop	Use an index to find matching inner tuples
Sort-merge	Sort inputs, then merge matching values
Hash join	Hash one input and probe using the join key

Key Takeaway

Join strategy selection depends on relation size, indexes, ordering, join condition, and available memory.